F1 RACE INTELLIGENCE
& STRATEGY SIMULATOR

Project plan, revision 2: full UI/UX, prediction, evidence and testing specification

Live analysis | Historical analysis | Strategy simulation

Solo project - built to be fun, interactive and explainable

Contents

Topic	Page
1. Problem statement	2
2. Tech stack	3
3. Architecture and data flow	4
4. Modules and functions	5-7
5. Interactive circuit map	8
6. Driver comparison screen	9
7. Historical analysis screens	10
8. Overtaking vs battle probability	11
9. Prediction uncertainty and reliability	12
10. AI evidence navigation	13
11. Saved analyses, search, notifications, settings	14
12. UX specification: structure and flows	15
13. UX specification: responsive, states, accessibility	16
14. Development flow: what first, what next	17
15. Dashboard wireframe	18
16. MVP: first 7 days	19
17. Versions: MVP to final	20
18. Testing plan 1: foundation and prediction	21
19. Testing plan 2: AI, simulation, UI	22
20. Timeline and risks	23

[+] *marks an addition or improvisation beyond your original idea. Numbers in diagrams are illustrative, not real model output.*

1. Problem statement

Fans watch timing screens and TV graphics but cannot easily see **why** a driver is losing time, how a strategy call changed a race, or what a different call would have done. This project is an interactive web app that combines sector-level comparison, an interactive circuit map, predictive insight with honest uncertainty, a question-answering AI whose answers link to visual proof, and a strategy sandbox for past races.

Goal: one app with three modes sharing a single analysis engine.

LIVE ANALYSIS

- Real-time dashboard + map
- Sector + mini-sector compare
- Tyres, pits, weather
- Battle + overtake odds
- Ask the AI anything

HISTORICAL ANALYSIS

- Explore and search any race
- Investigate, debrief, strategy
- AI: what went wrong / right
- Click evidence to jump there
- Save analyses

STRATEGY SIMULATION

- You are the strategist
- Only see what teams knew
- Call pits, tyres, push/save
- Compare with real race
- What-if: right call?

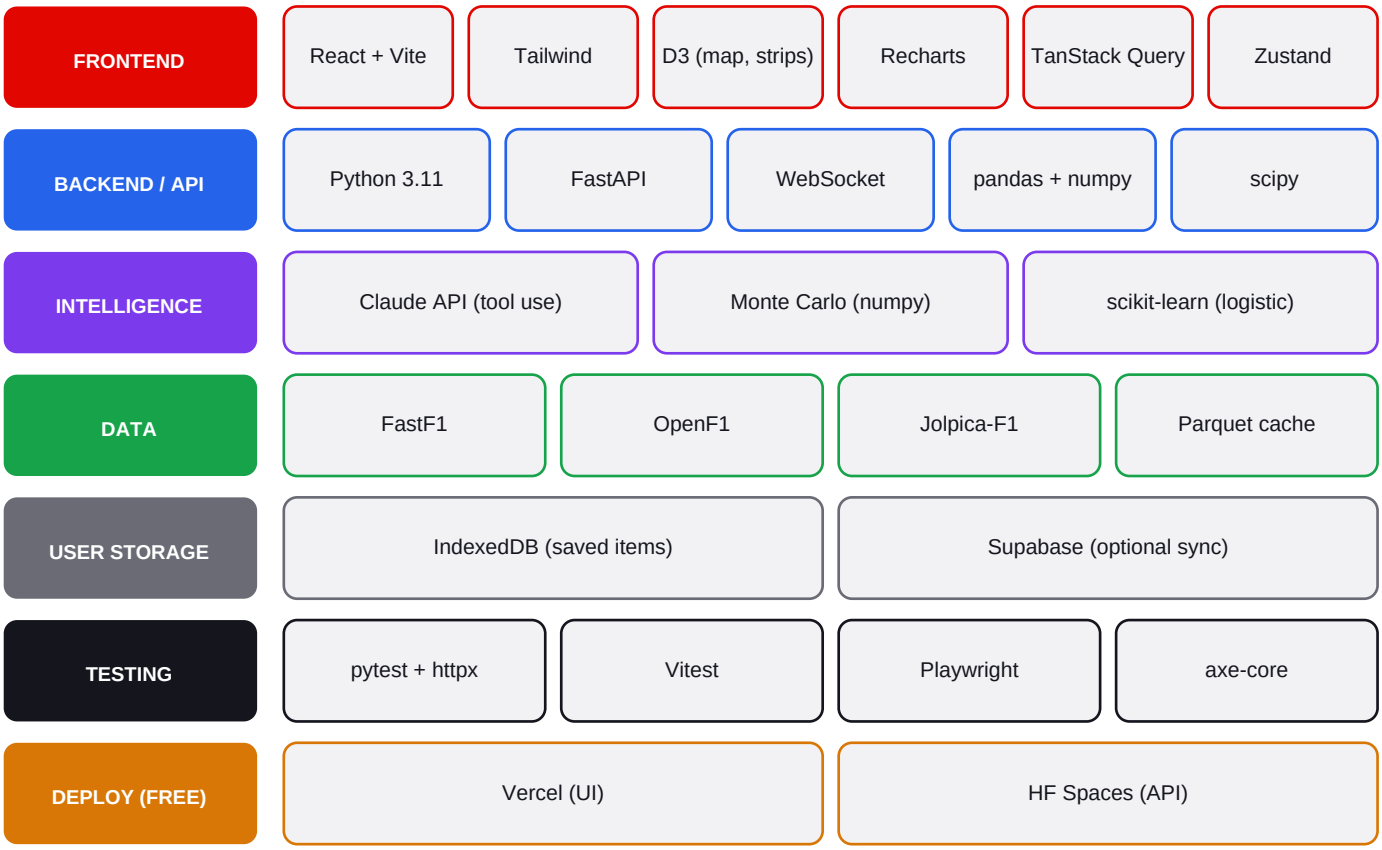
Reality checks that shape the design

Constraint	Consequence for the design
Official mini-sector times are not published (only colour status)	Compute them from telemetry by splitting each lap by distance [+]
Live data is paid and partly unavailable on OpenF1	Build on a replay engine first; point it at the live feed later [+]
LLMs can invent numbers	The AI never computes; it calls your functions, and a validator checks every number it states
Predictions are uncertain and models can be badly calibrated	Show ranges and confidence, and measure reliability by backtesting
Simulation outcomes are model estimates	Show outcome distributions, label as estimates, test against the real race
Free hosts have no persistent disk and can sleep	Saved items live in the browser (optional Supabase later); demo mode ships bundled races

Out of scope for now

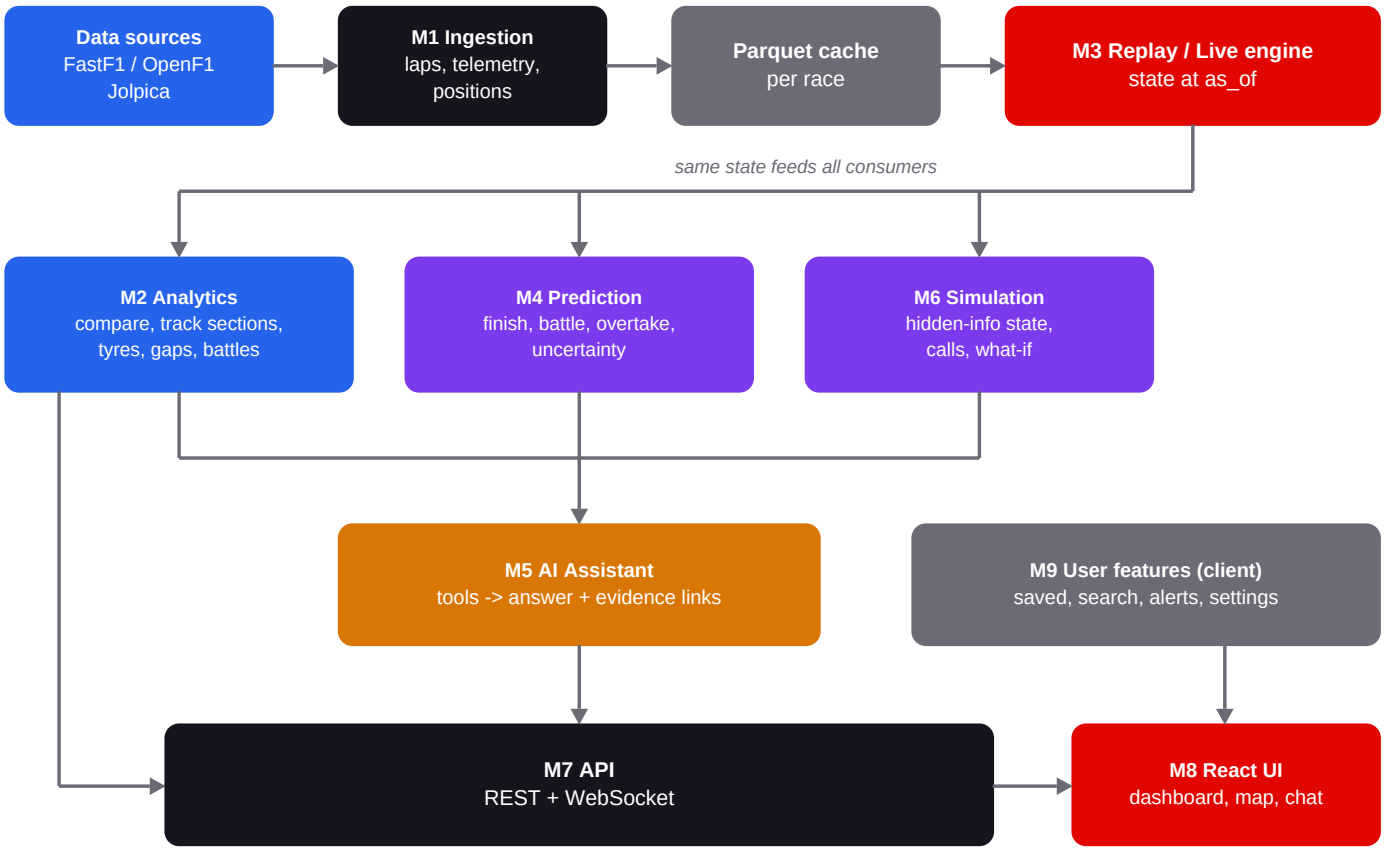
Team-radio audio, native mobile apps, push notifications from a server, production-scale hosting, exact reproduction of real team strategy software.

2. Tech stack



Choice	Why this one
React + Vite, Tailwind	You already know React; instant reloads; fast layout work
D3 + Recharts	D3 for the circuit map and heat strips; Recharts for standard line/bar charts
TanStack Query	Caching, retries and built-in loading/error states for every API call **[+]**
Zustand	One shared selection store (drivers, lap, section, time) so all panels stay linked **[+]**
FastAPI	Async, WebSocket support, auto-generated docs, pydantic schemas used in tests
FastF1 / OpenF1 / Jolpica	Telemetry and car positions (FastF1), live + 2023+ data (OpenF1), older results (Jolpica)
scikit-learn	Logistic models with calibration for battle and overtake probability **[+]**
IndexedDB, Supabase (optional)	Browser storage for saved analyses and settings; Supabase only if you later want sync across devices
pytest, Vitest, Playwright, axe-core	API/analytics tests, component tests, end-to-end flows, accessibility checks
Vercel + HF Spaces	Free; HF Spaces gives 16 GB RAM, enough for pandas + FastF1; free tiers can sleep when idle

3. Architecture and data flow



The as_of principle [+]: every function takes an **as_of** value (lap or time) and only sees data up to it. In live mode as_of is *now*; in replay it is the replay clock; in simulation it is the clock plus a filter for what your team could know. One engine serves all three modes, and the AI cannot leak future information.

Rule of thumb: only M1 talks to the internet. Everything else reads cached tables or the live adapter, so the app is testable offline. M9 runs entirely in the browser.

4. Modules and functions

M1 Data Ingestion | build first | the only module that touches the internet		
Function	**What it does**	**Returns**
load_session(year, gp, session)	Loads a session from FastF1, falls back to OpenF1	session object
get_laps(session)	Lap, sector times, compound, tyre age, position per lap	DataFrame
get_telemetry(session, driver, lap)	Speed, throttle, brake, gear, distance, time	DataFrame
get_positions(session, driver) [+]	Car X/Y over time, resampled to 5 Hz for smooth map animation	DataFrame
build_track_outline(session) [+]	Smoothed outline from a clean lap; sector, mini-sector and corner boundaries by distance	outline + sections
get_pits_stints(session)	Pit in/out laps, stop duration, stint compounds	DataFrame
get_weather_track_status(session)	Air/track temp, humidity, rain; flags, safety car / VSC	DataFrame
cache_race(year, gp)	Saves processed tables as Parquet so nothing is re-downloaded	files

M2 Analytics | the core of the app		
Function	**What it does**	**Returns**
compute_mini_sectors(tel, n=20) [+]	Splits the lap into n equal-distance segments, interpolates time at each boundary	n times per lap
compare_laps(drivers, laps, metrics, ref, sync)	One entry point for the comparison screen: laps by number, range, fastest, stint, last N clean; sync by lap or tyre age; any metric	table + series + chart spec
delta_trace(d1, d2, lap)	Cumulative time delta along lap distance	series
section_stats(section_id, drivers, laps)	Fastest driver, deltas, min speed, braking point in one track section	stats + traces
tyre_degradation(driver, stint)	Fuel-corrected lap-time slope per stint; flags the cliff	s/lap, cliff lap
gap_history(d1, d2)	Gap per lap and its closing rate	series
detect_battles(as_of)	Pairs within about 1 s with a closing rate above a threshold	list
pit_stop_analysis(driver)	Stop time vs field, undercut/overcut result, safety-car timing	verdict
time_loss_breakdown(driver)	Seconds lost to pit stop, traffic, tyre degradation, incidents vs a reference	factors in s

How mini-sectors are computed [+]

Mini-sector delta, D1 vs D2 on one lap (illustrative). Above = D1 slower, below = D1 faster



Check: the 20 mini-sector times of a lap must add up to the lap time (within about 0.05 s). This is your first test (page 21).

4. Modules and functions (continued)

M3 Replay / Live Engine | build right after M2		
Function	**What it does**	**Returns**
ReplayClock(play, pause, seek, speed) [+]	Drives race time for replay and simulation	current as_of
get_state(as_of)	Snapshot of positions, car X/Y, gaps, tyres, weather, flags at that moment	RaceState
visible_state(as_of, team) [+]	RaceState filtered to what that team could know	RaceState
LiveAdapter.poll()	Same interface fed by the live feed; swaps in without touching other modules	RaceState
stream_state()	Pushes state over WebSocket; supports resume from a given as_of after reconnect	events

M4 Prediction | V2 | every output carries an estimate, a range and a confidence level		
Function	**What it does**	**Returns**
finish_probabilities(state, n=2000)	Monte Carlo using pace, tyre degradation, pit plans, traffic, safety-car odds	P(position) distribution
battle_probability(d1, d2, H)	Chance the pair stays within 1 s for 2+ laps in the next H laps	p + range
overtake_probability(attacker, defender, H) [+]	Chance of a completed on-track pass: P_battle x P(pass | battle); excludes pit swaps	p + range + top factors
undercut_probability(attacker, defender)	Chance a pit-stop swap succeeds (kept separate from on-track passes)	p + range
pit_window(driver)	Likely lap range for the next stop	lap range
weather_trend(as_of)	Track temp / humidity trend and rain flag. An indicator, not a probability	trend label
confidence(prediction) [+]	High / Medium / Low / Not enough data from data sufficiency, horizon, volatility, model reliability	level
reliability_report(model) [+]	Backtest metrics (Brier, calibration, interval coverage) loaded from stored JSON	report

M5 AI Assistant | MVP (basic tools), grows each version		
Function	**What it does**	**Returns**
tool registry	8-12 analysis functions exposed to the LLM through tool calling	tool schemas
answer_question(q, as_of) [+]	Tool-calling loop; fixed output: short answer, claims, evidence list, 2-3 key numbers	answer object
build_evidence(tool_results) [+]	Turns tool results into typed evidence links (lap, section, event, compare view)	evidence[]
validate_answer(answer, tool_results) [+]	Rejects numbers or laps not present in tool results; checks every evidence link resolves	ok / retry
explain_time_loss(d1, d2, laps)	Combines mini-sector delta, tyre age, traffic, track status	evidence bundle
debrief_driver(driver)	What went wrong or right, using time_loss_breakdown and events	verdict + factors
auto_insights(state) [+]	Rule-based triggers produce alert cards (feeds M9 notifications)	alerts

Guardrail: the AI states its confidence and says 'cannot tell from the data' for things telemetry cannot show (damage, engine modes, driver intent).

4. Modules and functions (continued)

M6 Simulation | V3

Function	What it does	Returns
start_simulation(race, team)	Begins at the chosen lap with the hidden-information state	sim session
apply_call(lap, action)	Pit now, compound, stay out, push or save tyres; only for current or future laps	updated plan
simulate_forward(plan, until_lap, seed)	Advances with pace model, tyre degradation, pit loss, traffic, safety-car odds; seeded for determinism	RaceState
simulate_distribution(plan, n=500)	Runs many seeded variations to give an outcome range, not one result	distribution
compare_with_actual()	Your finish vs the real one, stint by stint	comparison
what_if(strategy)	Re-runs the race with an alternate strategy	result
ai_race_engineer(q) **[+]**	Advice using only information visible at that moment; post-race debrief	text + evidence

M7 API Layer

Endpoint	Purpose
GET /races, /races/{id}/overview	Race list with filters; overview data (result, race trace, strategy, events)
GET /races/{id}/state?as_of=	RaceState snapshot including car positions
GET /races/{id}/track	Track outline, sections, corners
WS /stream/{id}	Live or replay state stream; commands: play, pause, seek, speed, resume
POST /compare	Comparison for chosen drivers, laps, metrics, reference, sync
GET /races/{id}/predictions?as_of=	Battle, overtake, finish odds with ranges, confidence, reliability
POST /ask	AI question with as_of; returns answer, claims, evidence links
POST /sim/start, /sim/call; GET /sim/result	Simulation lifecycle

M9 User Features (client side) | details on page 14

Function	What it does
saveAnalysis / loadAnalysis(id)	Stores and restores a full view state with pinned AI answers and notes
searchIndex.query(text)	Command palette over races, drivers, laps, events, saved items
alertEngine.evaluate(state)	Applies the user's alert rules, thresholds and cooldowns to auto_insights output
settings.get / set	Theme, units, defaults, followed drivers, spoiler-free mode

5. Interactive circuit map

The map is a live view of the race state: cars move with the replay clock, and the track itself is selectable, so it becomes an entry point into the comparison and AI features.



Click or drag along the track to select a section. Squares = second car of a team.

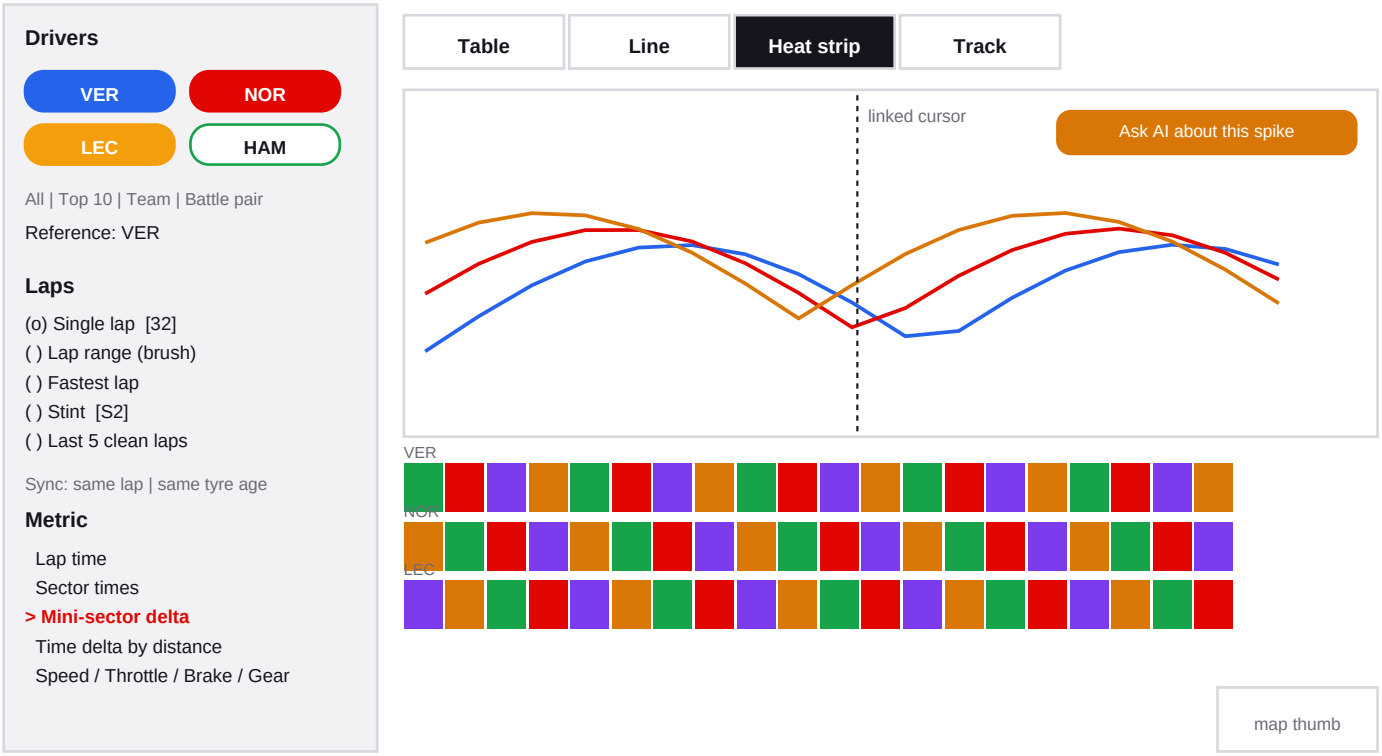
Feature	Behaviour
Moving markers	Positions from get_positions at 5 Hz; the client interpolates every animation frame. Team colour + 3-letter code; second car of a team is a square (colour-blind safe)
Marker states	Running; in pit lane (docked at pit entry); retired (grey X); safety-car queue (amber ring); selected (white ring); compared pair (thin line with gap label)
Hover / click	Hover: tooltip with position, gap, tyre and age, speed. Click: adds the driver to the comparison selection
Selectable sections	Layers: 3 sectors, 20 mini-sectors, corners (T1...). Click one, or drag along the track to select a range. Opens the section card (fastest driver, deltas, speed/brake trace) with 'Ask AI about this section'
Overlays	Colour the track by: fastest driver per mini-sector (dominance map), delta D1 vs D2, speed, braking zones **[+]**
Zoom / pan	Wheel or pinch; double-click resets; the selection persists while zooming

Data and performance

Topic	Decision
Track outline	Built from X/Y of a clean lap, smoothed; mini-sectors placed by cumulative distance so they match M2 exactly
Rendering	SVG for the track, markers moved through refs and transforms (not React state) to keep 60 fps with 20 cars; reduced-motion setting switches to step updates
Missing data	No X/Y for a session: hide the map and show a lap-distance bar. Live car positions are the weakest live feed, so the MVP map runs in replay

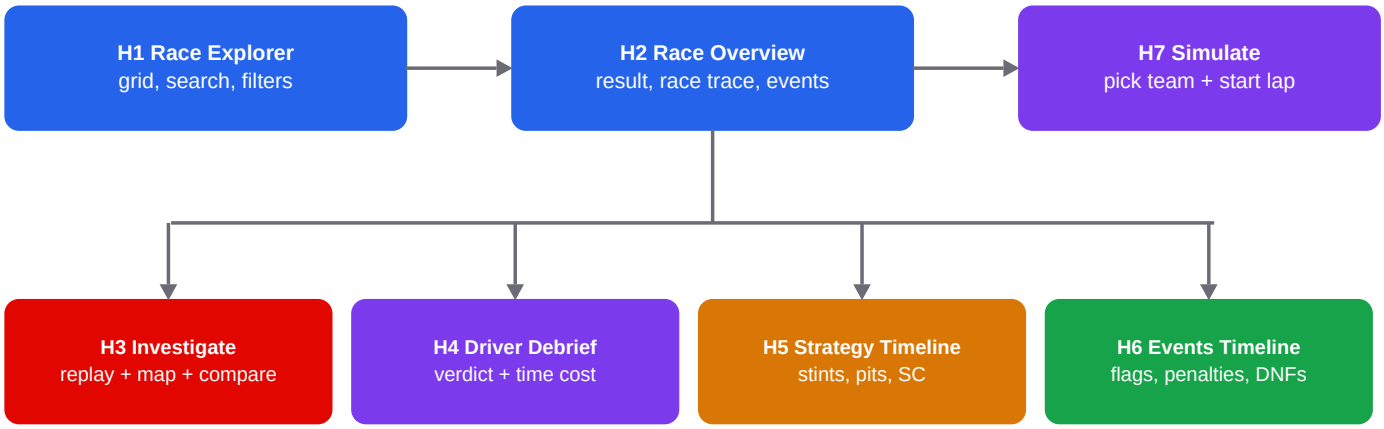
6. Driver comparison screen

A dedicated screen for choosing **who**, **which laps** and **which metric**, with several views of the result. Selections are shared with the map and the dashboard.



Control	Options
Drivers	Chips with multi-select; presets All, Top 10, Same team, Current battle, Followed. Chart overlays limited to 6 drivers for readability; table and heat strip handle all 20
Reference driver	Baseline for every delta (default: race leader or first selected)
Laps	Single lap, lap range (brush on a lap-time chart), fastest lap, stint, last N clean laps. Sync by lap number or by tyre age **[+]**
Metrics	Lap time, sector times, mini-sector delta, time delta by distance, speed, throttle, brake, gear
Views	Table, line chart, heat strip, track overlay (opens on the map)
Filters	Clean laps only by default (excludes in/out laps and safety-car laps); fuel correction on/off
Interactions	Linked cursor across charts and map; click an anomaly for 'Ask AI about this' (prefilled question and context); save view; export PNG

7. Historical analysis screens



Ask AI panel, search and alerts available on every screen

Evidence chips and click-throughs jump between H3, H5 and H6 (e.g. click a stint to open it in Investigate)

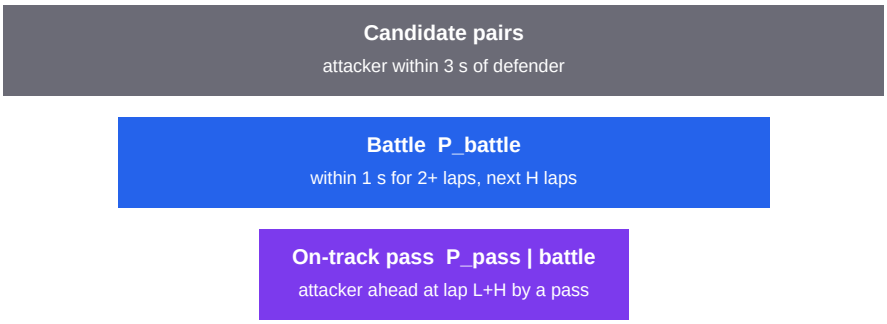
Spoiler-free mode hides results and winner on H1/H2 until the user reveals them.

Screen	Purpose and key content
H1 Race Explorer	Season/race grid or list, search, filters (year, circuit, dry/wet/mixed, safety car, winner), offline badge for bundled races, spoiler-free toggle
H2 Race Overview	Result, grid vs finish, position-per-lap chart, race trace (gap to leader per lap), strategy summary, key events strip; buttons to Investigate, Debrief, Simulate
H3 Investigate	The full dashboard (page 18): replay clock, map, compare, tyres, pits, weather, battles
H4 Driver Debrief	Verdict card (fine race / lost time to pit stop, strategy, traffic, incident) plus factors with estimated seconds lost, each with evidence chips. 'Nothing went wrong' appears when unexplained loss is below a threshold
H5 Strategy Timeline	Stint bars per driver coloured by compound, pit laps, SC/VSC/red-flag shading, undercut/overcut markers; click a stint to open it in Investigate
H6 Events Timeline	Flags, safety cars, penalties, DNFs, fastest laps, weather changes, pit stops; click an event to jump the replay to that moment
H7 Simulate	Choose team and start lap, then enter simulation mode (topic 12 flow B)

Debrief seconds are estimates from time_loss_breakdown against a stated reference (e.g. the field's median stop); the screen shows the reference used.

8. Overtaking vs battle probability

These are two different questions and the UI shows both. **Battle probability**: will two cars fight? **Overtake probability**: will the attacker actually get past on track?



Overtake probability

P_pass = P_battle x P_pass | battle

0.72 x 0.43 = 0.31

range 22% - 41%, confidence: Medium

(illustrative numbers)

	Battle probability	Overtake probability
Question	Will the pair stay within about 1 s for 2+ laps in the next H laps (default H = 5)?	Will the attacker be ahead at lap L+H through an on-track pass?
Main drivers	Gap and closing rate, clean-lap pace delta, pit windows	Pace delta in clean air, tyre age and compound delta, top-speed delta, DRS / overtake-mode availability (varies by era), track overtaking difficulty, defender's pit window, laps remaining, traffic ahead of defender
Excludes	Safety-car and VSC laps	Pit-stop swaps (those are undercut_probability), lap 1, SC/VSC laps
Method	Logistic model on pair features	P_battle x P(pass | battle), logistic model trained on past races **[+]**
Shown as	Percentage + range + confidence	Percentage + range + confidence + top 3 contributing factors

Training data and method

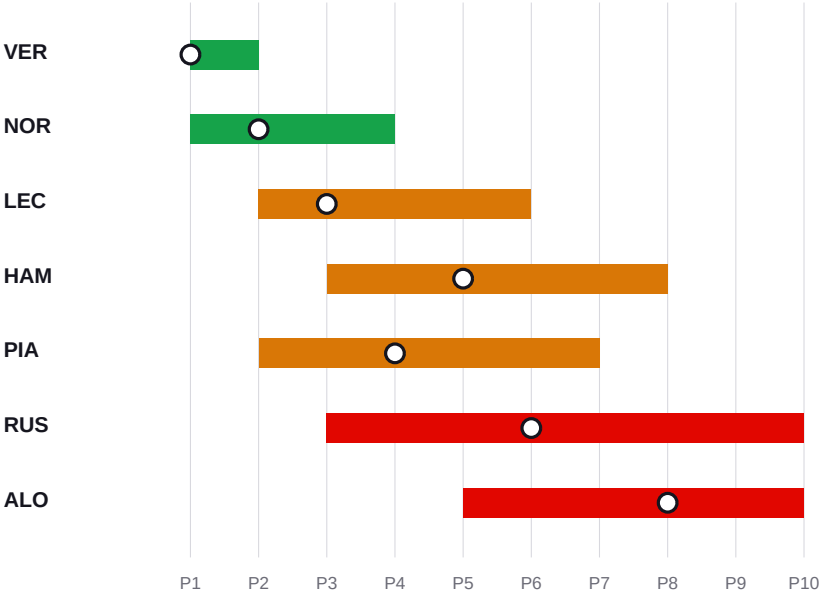
Samples: every attacker/defender pair within 2 s at the start of a green-flag lap; label: on-track pass within the next H laps. Validation: leave-one-race-out so the model is never tested on a race it trained on. Calibration: Platt or isotonic scaling; ranges from bootstrapped models. Per-circuit overtaking difficulty comes from the historical pass rate of the cached races; with few races the UI shows the sample size.

In the UI: a battle card shows 'Battle 72%' and 'Pass 31% (22-41%)', the gap sparkline and factor chips such as '+ tyre advantage', '- hard to pass here'. An alert fires when battle probability crosses the user's threshold while rising.

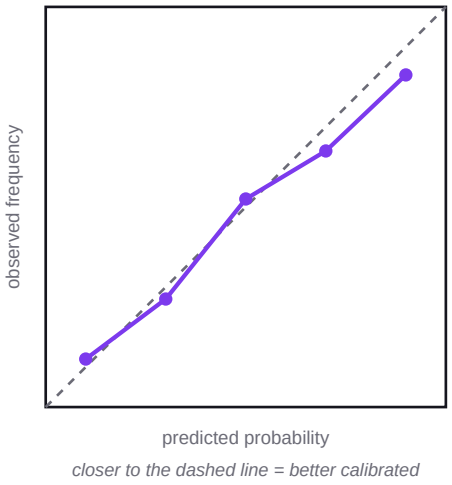
9. Prediction uncertainty and reliability

Three rules: never show a bare number (estimate, range, confidence together); measure reliability on past races; refuse to predict when data is insufficient.

Expected finish with 80% range, by confidence (illustrative)



Model reliability (illustrative)



High | Medium | Low (faded) | No number shown

Prediction	Uncertainty shown	How it is computed	Reliability check
Finish order	P(win / podium / points); expected position with 80% range	Percentiles of the Monte Carlo runs	Brier score, ranked probability score, interval coverage
Battle, overtake	Probability with bootstrap range	Ensemble of bootstrapped logistic models	Brier, calibration error, reliability diagram
Pit window	Lap range (10th-90th percentile)	Simulation with random tyre degradation	Hit rate of the range on past stops
Simulation what-if	Outcome distribution, not one result	Many seeded runs with random SC, traffic, degradation	Fidelity test (page 22)
Rain	Trend indicator only	Temperature / humidity trend and rain flag; no probability model	Labelled low confidence

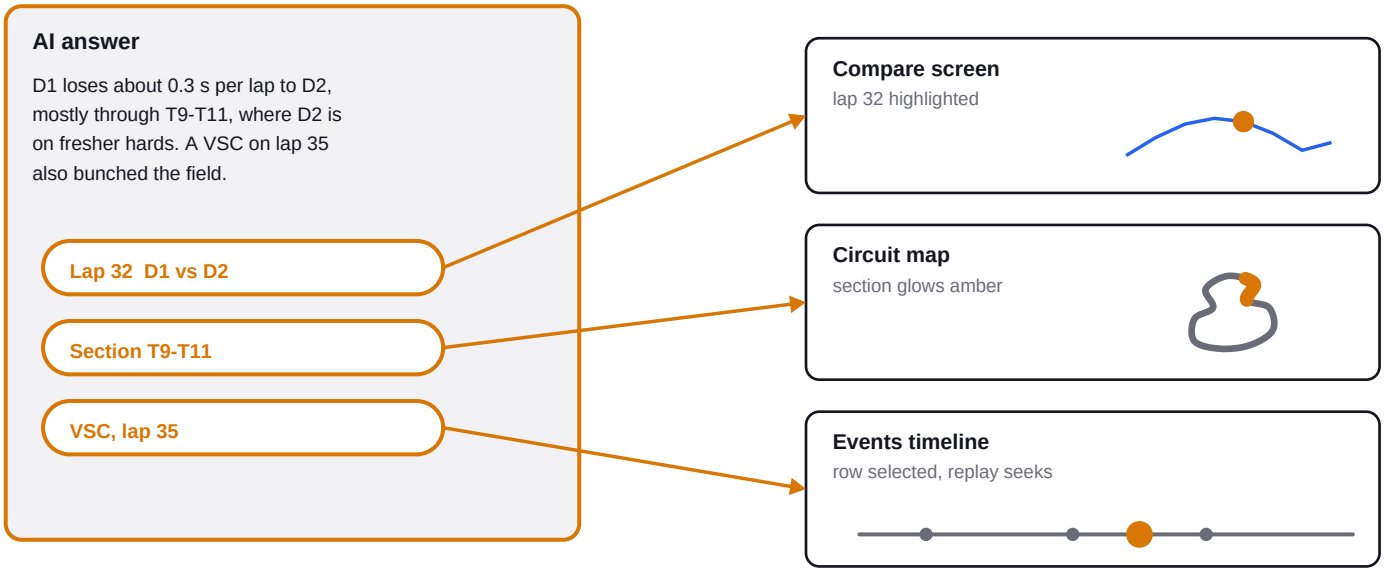
Confidence level (weakest link wins)

Factor	High	Medium	Low / blocked
Data sufficiency	5+ clean laps in the current stint	3-4 clean laps	under 3: no number shown
Horizon	up to 3 laps	4-8 laps	beyond 8 laps
Volatility	stable green running	recent SC/VSC or pit cycle	rain, red flag, damage reported
Model reliability in this bucket	calibration error below target	slightly above	poor, or too few samples

Reliability panel: 'When the model said 60-70%, it happened 64% of the time (n = 182). Brier 0.17 vs baseline 0.21' (numbers come from the stored backtest JSON; example values only). With few cached races the sample size is shown prominently.

10. AI evidence navigation

Every AI answer is a structured object. Each claim carries evidence links; clicking a link opens the exact chart, lap, section or event that proves it.



Item	Specification
Evidence types	lap, lap_range, driver_compare (drivers, metric, laps), track_section, race_event, pit_stop, stint, chart view
Click behaviour	Sets the shared URL state (screen, race, as_of, drivers, metric, section, lap, highlight). The destination renders the highlight (pulse on the lap, glowing section, selected event row) and shows 'Back to answer'. Links are shareable
Preview on hover	A small thumbnail of the target chart [+]
Live mode	Past evidence opens in a review pane; the live clock keeps running, with a 'Back to live' button [+]
Historical mode	The replay seeks to the evidence time
Validation	The backend resolves every link against real data (lap exists, section valid, time not in the future) before sending; broken links are dropped and the answer is flagged

Answer object (shape)

```
{ "answer": "D1 loses ~0.3 s/lap to D2...", "confidence": "medium", "claims": [ { "text": "...mostly in T9-T11",
"evidence": ["e1","e2"] } ], "evidence": [ { "id": "e1", "type": "track_section", "params": { "section": "T9-T11",
"drivers": ["D1","D2"], "laps": [30,34] }, "label": "Section T9-T11" }, { "id": "e2", "type": "race_event", "params": {
"event": "VSC", "lap": 35 }, "label": "VSC, lap 35" } ] }
```

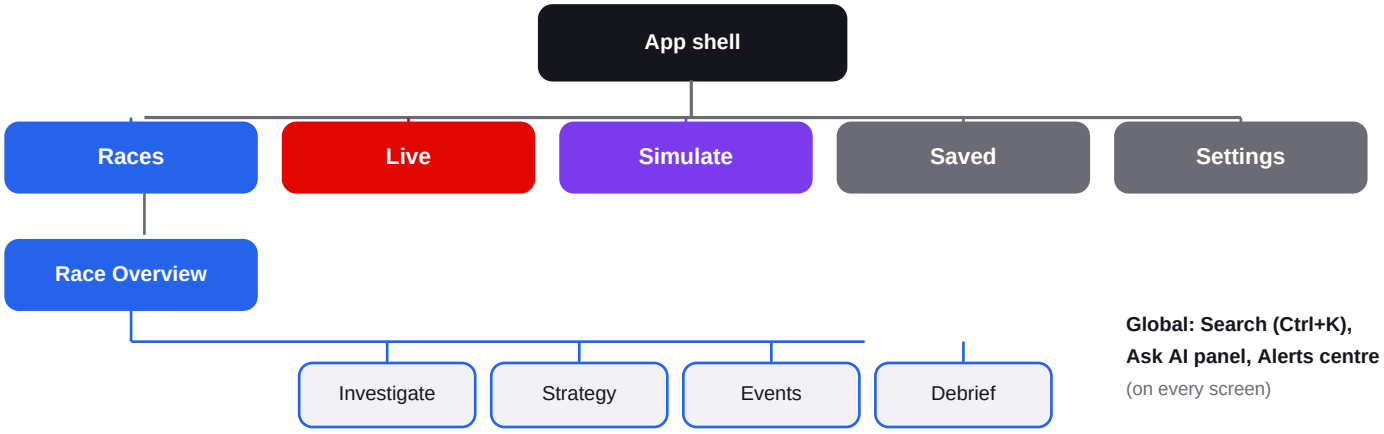
11. Saved analyses, search, notifications, settings

Storage: all four live in the browser (IndexedDB, small items in localStorage) because free hosting has no persistent disk and the plan has no accounts. Optional later: Supabase sign-in to sync across devices.

Feature	Specification
Saved analyses	A snapshot stores race, mode, as_of, drivers, laps, metric, view, section, pinned AI answers (with evidence) and notes. Library screen with cards: open (restores the exact view), rename, duplicate, delete, export JSON, copy share link (URL state, works for bundled races). Saved simulations include the calls and result
Search	Command palette (Ctrl/Cmd+K) over races, drivers, laps ('lap 32'), events ('safety car'), saved analyses and commands; fallback entry 'Ask AI: ...'; recent searches. Client-side index (Fuse.js) built when a race loads
Notification types	Battle forming, overtake likely, tyre cliff, pit window opens, SC/VSC/red flag, weather change, rival pits, fastest lap
Delivery	In-app toast plus an alert centre (history, unread badge). Optional browser notification while the tab is open (permission requested on first enable). No server push: it would need a backend service and service worker
Alert controls	Per-type toggles, followed drivers only, threshold sliders (e.g. battle above 60%), cooldown of one alert per pair per 3 laps to avoid spam **[+]**. In replay, alerts follow the replay clock
Settings: appearance	Dark / light / system, colour-blind palette, reduced motion
Settings: units and data	km/h or mph, C or F, time format; mini-sector count (10 / 20 / 30), clean laps only, fuel correction
Settings: defaults	Landing screen, replay speed, followed drivers/team, AI answer detail (short / detailed)
Settings: spoilers and privacy	Spoiler-free mode hides results until revealed **[+]**; clear saved data; clear cache

12. UX specification: structure and flows

Navigation structure

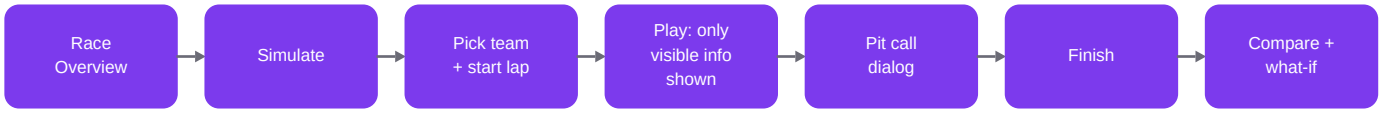


Core interaction flows

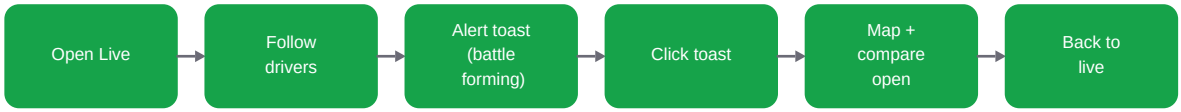
A Why is D1 losing time?



B Make strategy calls

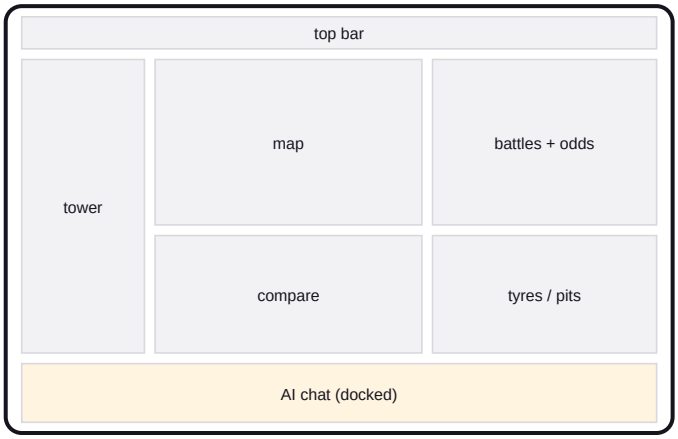


C Follow a live race

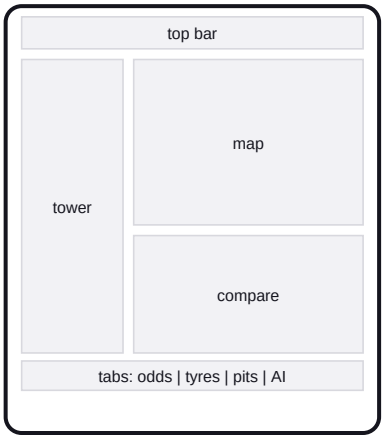


Principles: one shared selection (drivers, lap, section, time) drives every panel (linked brushing); the default view is simple and detail appears on hover or expand; every chart can hand a question to the AI with context already filled in; every AI claim can be traced back to a chart.

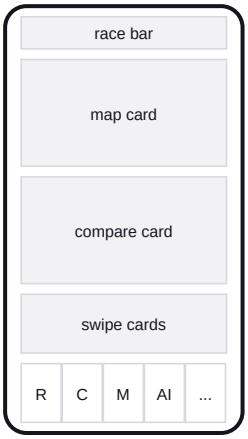
13. UX specification: responsive, states, accessibility



Desktop >= 1200 px: 3 columns



Tablet 768-1199: 2 cols



Mobile < 768: 1 col

Situation	UX behaviour
First load of a race	Skeletons; progressive loading: tower first, then charts; telemetry fetched on first comparison with progress text ('Loading telemetry 3/20')
Cold start of free host	Splash 'Waking the server' with progress; free hosts can take up to about a minute
Empty states	No drivers selected: hint plus presets. No saved analyses: call to action. No race matches filters: reset-filters button
Missing data	Grey cell with tooltip 'No telemetry for this lap'; map hidden if no positions
API error	Inline error card with Retry; last good data stays visible; technical detail expandable
WebSocket drop	Banner 'Reconnecting...' with backoff; resumes from last as_of; falls back to polling after 3 failures
Stale live data	'Last update 12 s ago': amber after 10 s, red after 30 s
AI loading / failure	Typing indicator with tool steps ('Comparing mini-sectors...'); on failure 'Could not answer, charts still work' with retry; 30 s timeout
Rate limit	Friendly message with the reset time
Low-confidence or insufficient data	Faded estimate with 'Low confidence' tag, or 'Needs 5 clean laps' instead of a number

Keyboard and accessibility

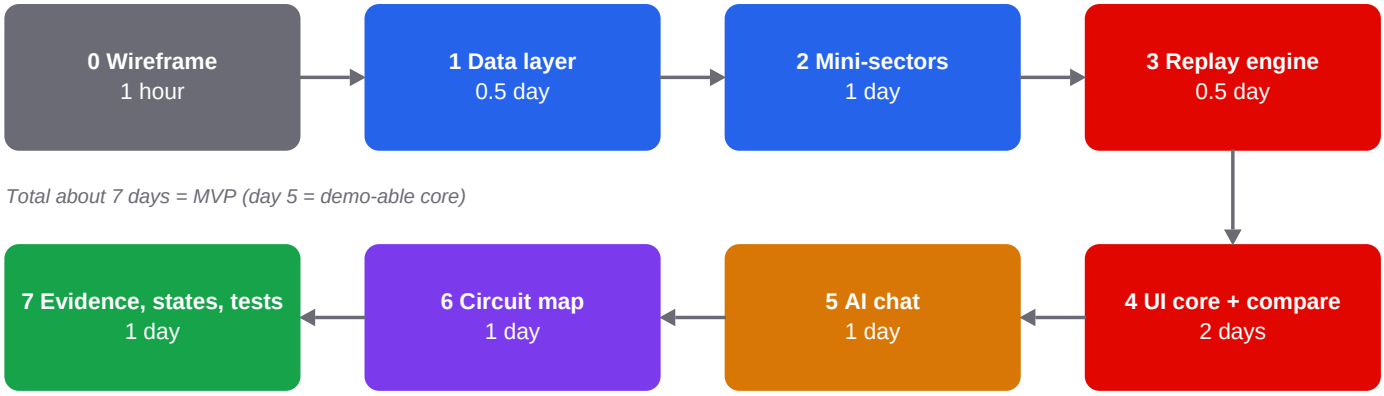
Area	Specification
Shortcuts	Space play/pause; Left/Right step lap; 1-5 speed; C compare; M map; A ask AI; / or Ctrl+K search; Esc clear selection
Accessibility	Colour-blind safe palette; meaning never by colour alone (shapes, labels); WCAG AA contrast; visible focus; aria-live for alerts; reduced motion; chart data available as tables; touch targets at least 44 px

14. Development flow: what first, what next

Should you design the UI first? Not fully. Do a one-hour paper wireframe (page 18), then data, then analysis, then build the UI on **real data**, adding the circuit map after the comparison screen works. Each step produces something you can see.

Why this order

Reason	Explanation
The UI depends on the data's shape	Mini-sector times exist in no feed, and car positions need their own pipeline. Build them before deciding what a chart or map can show.
UI on fake data gets rebuilt	Mock data hides real problems (missing laps, pit-lap outliers, telemetry gaps).
A wireframe is cheap, a redesign is not	One hour fixes the layout and which numbers matter.
Replay engine early **[+]**	Once a race replays, you can test any feature on any day with no live race and no paid feed.
Map after compare **[+]**	The map reuses the shared selection and the section data from M2, so it is faster to build once those work.

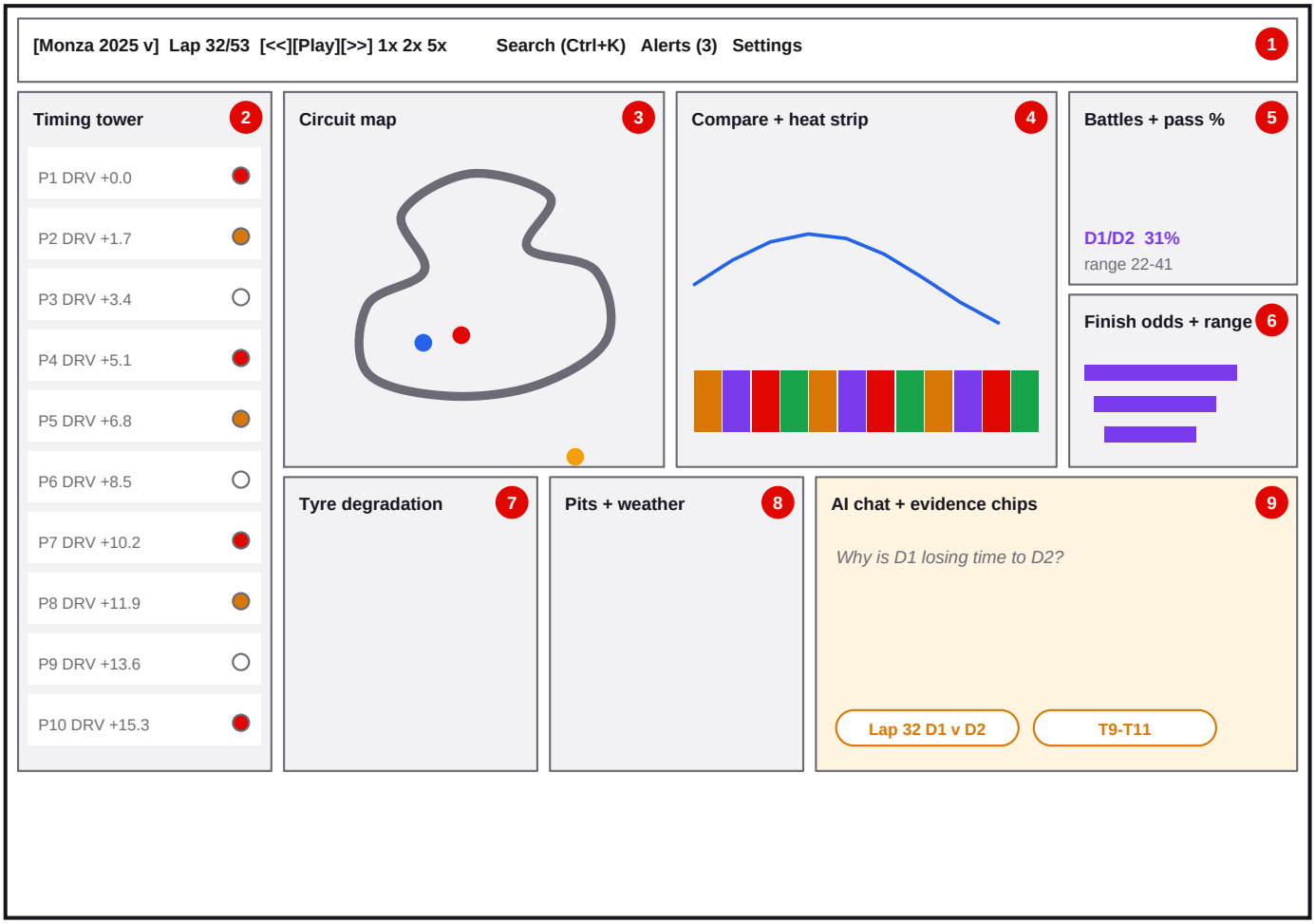


Total about 7 days = MVP (day 5 = demo-able core)

Rule: finish and check each step before the next. 'Done when' criteria are on page 19.

15. Dashboard wireframe

Sketch this on paper first, adjust, then use it as the build target.



#	Panel	Fed by	Built in
1	Race bar, playback, search, alerts, settings	ReplayClock, M9	MVP (search, alerts: V4)
2	Timing tower	get_state	MVP
3	Circuit map	get_positions, build_track_outline	MVP (sections: V2)
4	Compare + heat strip	compare_laps	MVP
5	Battles + overtake %	detect_battles, overtake_probability	V2
6	Finish odds with ranges	finish_probabilities	V2
7	Tyre degradation	tyre_degradation	MVP
8	Pits + weather	M1, M2	MVP
9	AI chat + evidence chips	answer_question	MVP

16. MVP: first 7 days, step by step

Day	Do	Why	Done when
1	Wireframe (1 h). Repo, FastF1, load one completed race, cache as Parquet.	Fixes layout; proves you can get real data first.	Lap times match official results.
2	compute_mini_sectors, compare_laps. Then ReplayClock + get_state.	Mini-sectors are the signature feature and biggest unknown; the clock makes everything testable.	Mini-sector sum equals lap time (about 0.05 s).
3	FastAPI + WebSocket. React shell: playback bar, timing tower, shared selection store.	First time the full chain runs: data to browser.	Press play and watch positions change.
4	Comparison screen: selectors, line chart, heat strip. Tyre, pit, weather panels.	The comparison screen is the core user value.	Compare any 2 drivers on any lap range.
5	AI tools (5-6), tool-calling loop, chat UI, answer format. Holiday milestone.	Tools wrap functions you already tested.	'Who is P2?' and 'why is D1 slower than D2?' answer with numbers and a chart.
6	Circuit map: outline, moving markers, hover tooltip, click to select driver.	Needs positions and shared selection from days 3-4.	20 markers move smoothly at 5x speed.
7	Evidence chips (lap and compare links), loading/error states, core tests, bundle 3 races, deploy, README.	Chips make the AI believable; tests protect the base for V2.	A friend can open the link and use it.

If you fall behind, cut in this order

1) weather panel 2) pit panel 3) deployment (run locally, record a screen demo) 4) number of bundled races. Never cut the replay engine, mini-sector comparison, the AI chat or the shared selection model.

17. Versions: MVP to final

each version is demo-able on its own

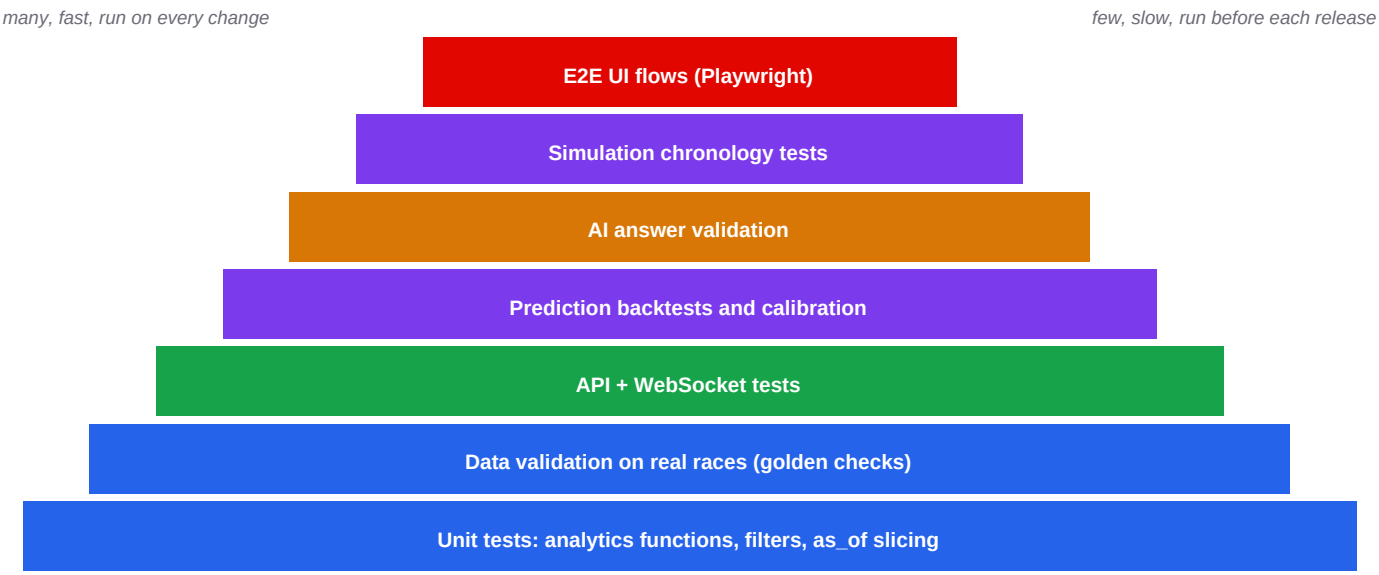


Version	Adds	Time
MVP	Replay dashboard, timing tower, driver comparison screen, tyre/pit/weather panels, AI chat with chart evidence chips, interactive map with moving markers and hover, loading/error basics, core tests, 3 bundled races, free deploy	7 days (day 5 = demo-able core)
V2 Insights and investigation	Battle and overtake probability, finish odds with ranges and reliability panel, auto-insights, historical explorer / overview / strategy / events / debrief screens, selectable track sections and overlays, full evidence navigation, prediction backtests	+7 days
V3 Simulation	Hidden-information state, call-making UI, forward simulation and outcome distributions, compare with actual, what-if, AI race engineer, chronology and fidelity tests	+5 days
V4 Product and live	Saved analyses, search, notifications, settings, responsive and mobile pass, states audit, live adapter and live tests, model tuning, docs and demo video	+8 days

Total: about 27 working days, about 160 hours, assuming about 6 focused hours a day with AI coding help. Plan on roughly double if you are learning FastF1, D3, WebSockets and model evaluation from scratch. Your 5 holiday days reach the day-5 milestone; the MVP completes with 2 more days.

Each version is deployable and demo-able. Stopping at any version still leaves a finished project.

18. Testing plan 1: foundation and prediction



Unit, data and API tests

Area	Checks	Pass criterion
Analytics units	Clean-lap filter excludes in/out and SC laps; as_of slicing; gap symmetry; section mapping	All pass, run on every change
Data validation (real race)	Mini-sector times sum to lap time; lap counts and final order match Jolpica; 5 golden facts (winner, fastest lap, a pit lap...)	98%+ of clean laps within 0.05 s; all others explained
API (pytest + httpx)	Every endpoint returns schema-valid JSON on a small fixture race stored in the repo; 4xx on unknown race or as_of beyond race end; 429 with Retry-After on rate limit	100% pass; offline
WebSocket	Connect, play, pause, seek, speed, drop and resume from last as_of	State stream stays in order
Performance	State endpoint and compare endpoint timing on the fixture	Budgets set after first measurement (start: 200 ms / 500 ms)

Prediction evaluation

Item	Specification
Protocol	Leave-one-race-out backtest over all cached races (about 10 or more for meaningful numbers); snapshots every 5th lap to limit overlap; green-flag situations and SC situations reported separately
Finish order	Brier score for win / podium / points and ranked probability score; baselines: current-order persistence and grid order
Battle, overtake	Brier, AUC, calibration error and reliability diagram; baseline: constant base rate
Ranges	80% intervals should contain the truth about 70-90% of the time; pit-window hit rate
Acceptance	Must beat the baseline on Brier; calibration error target set after the first backtest (start at 0.05 and adjust to what the data supports)
Regression guard	Backtest results stored as JSON (feeds the UI reliability panel); CI fails if Brier worsens by more than 5%

19. Testing plan 2: AI, simulation, UI

AI answer validation

Check	Method	Pass criterion
Tool selection	Golden set of about 40 questions (basic, analytical, unknowable) with expected tools and arguments	90%+ correct tool
Numeric grounding	Validator extracts every number, driver and lap from the answer and requires it in the tool results	100%; else retry once, then template answer
Evidence resolution	Every evidence link resolves to an existing lap, section or event	100%
Chronology	Ask at lap 10 about lap 30 events	Answers 'not known yet'
Unknowable questions	Damage, engine mode, driver intent	Says 'cannot tell from the data'
Consistency	Same question twice	Identical numbers
Human review	20 answers rated for correctness and clarity before each release	80%+ rated good

Simulation chronology and fidelity

Test	Method
No future reads	Wrap every data accessor in an as_of guard that raises if a timestamp is later than as_of; run a full simulation under the guard
Visibility rules	A table lists what is visible when (public timing; own team data; rival tyre compound once observed). Test that rivals' planned stops, future safety cars and weather never appear in visible_state
Monotonic time	Lap number and race time never decrease; calls only apply to the current or future laps
Determinism	Same seed gives the same result; different seeds give the distribution
Fidelity	Feed the real strategy into the simulator and compare with the real result; report mean position error. If it is poor the mode is labelled indicative
Sanity	Known effects hold: pitting under SC costs less than under green; fresher tyres are faster on a given stint

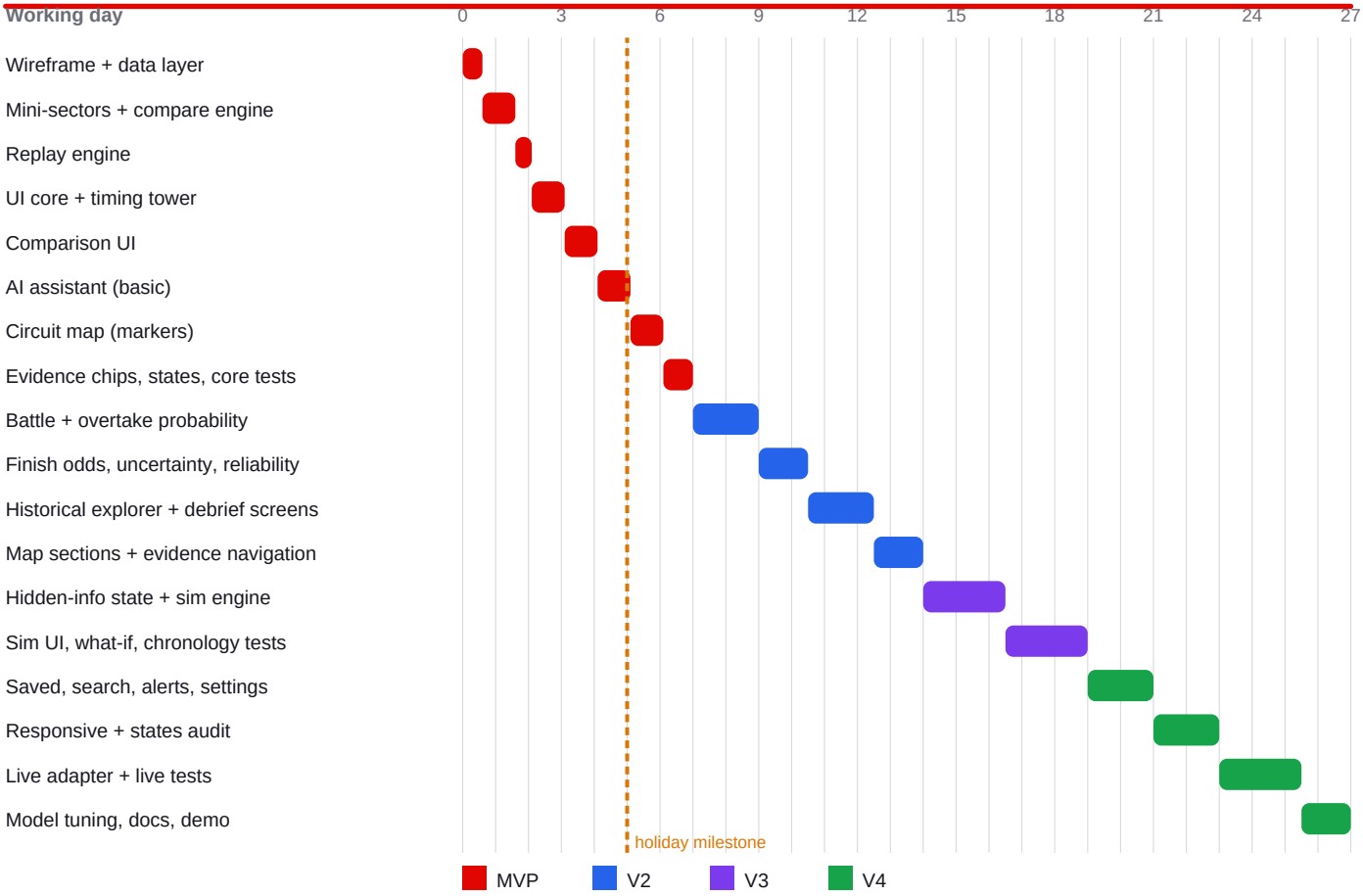
UI, accessibility and live

Area	Method
E2E (Playwright)	Open race, play, select drivers, compare; click an evidence chip and see the highlight; save analysis, reload, restore; run a simulation to the end; mocked network failures for error states
Quality	axe-core with no serious violations; screenshots at 3 breakpoints; Lighthouse performance budget
Live	Dry run with a recorded stream replayed as fake live; test on a practice session before a race; 2-hour soak test

When tests are added

Version	Tests added
MVP	Analytics units, data validation, API and WebSocket tests, 15-question AI golden set, 3 E2E flows
V2	Prediction backtests and reliability JSON, evidence-resolution tests, full AI set
V3	Chronology suite and fidelity test
V4	Accessibility and responsive checks, live soak test, CI regression gates

20. Timeline and risks



Risks

Risk	Mitigation
Data API limits or outages	Cache every race as Parquet; the app reads the cache only
Live feed gaps (positions, intervals)	Compute gaps from lap and sector data; develop on replay; go live last
Mini-sector or map misalignment	Place mini-sectors by the same distance axis in M2 and the map; validate sums against lap time
Predictions poorly calibrated with few races	Show sample size and confidence; start with simple models; label low-confidence output
AI inventing facts	Tools only, numeric validator, evidence resolution check
Simulation looks too exact	Show ranges, label as estimate, run the fidelity test
Scope creep (this plan is about 27 days)	Finish each version first; use the cut list on page 19; V4 items are optional
Free hosting sleeps or limits RAM	Cold-start splash, demo mode with bundled races, HF Spaces for the API